

Seminar 4

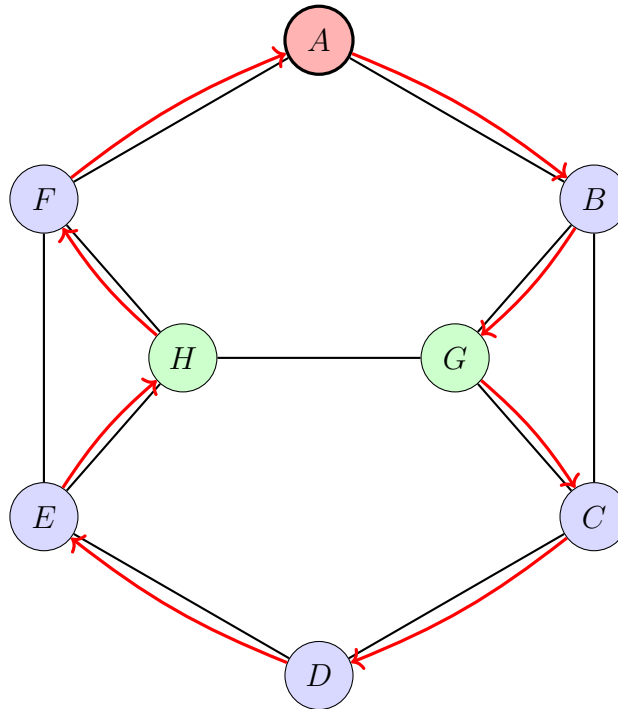
Lanțuri și cicluri Hamiltoniene

Asist. univ. dr. Mihai Tudor

9 martie 2026

1 Motivație istorică

Conceptul de ciclu hamiltonian apare în 1857, când matematicianul irlandez **Sir William Rowan Hamilton** a inventat un joc de puzzle numit *Icosian Game*. Jocul consta dintr-un dodecaedru pe ale cărui vârfuri erau marcate orașe importante ale lumii. Provocarea: găsiți un traseu care vizitează fiecare oraș exact o dată și revine la punctul de plecare, deplasându-vă numai pe muchiile dodecaedrului.



Ciclu hamiltonian: $A \rightarrow B \rightarrow G \rightarrow C \rightarrow D \rightarrow E \rightarrow H \rightarrow F \rightarrow A$

Nodul de start și final: A (marcat cu roșu)

Ciclu hamiltonian marcat cu roșu pornește din A , vizitează cele 8 vârfuri exact o dată fiecare și se întoarce în A .

Deși problema pare similară cu cea a grafurilor euleriene (Seminarul 2), există o diferență fundamentală: **nu există o condiție simplă, de tipul gradelor, care să garanteze existența unui ciclu hamiltonian**. Problema este, în general, NP-completă.

2 Definiții fundamentale

Definiție 2.1. Fie $G = (V, E)$ un graf neorientat.

- Un **lanț hamiltonian** este un drum simplu care trece prin fiecare vârf din V exact o dată.
- Un **ciclu hamiltonian** este un ciclu simplu care trece prin fiecare vârf din V exact o dată și revine la vârful de start.
- Un graf care admite un ciclu hamiltonian se numește **graf hamiltonian**.

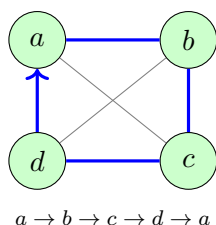
- Un graf care admite un lanț hamiltonian (dar nu ciclu) se numește **graf semi-hamiltonian**. (În această definiție, grafurile hamiltoniene nu sunt considerate semi-hamiltoniene.)

Observație 2.1. Comparație esențială cu grafurile euleriene:

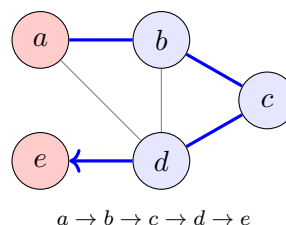
	Eulerian	Hamiltonian
Parcurge	fiecare muchie exact o dată	fiecare vârf exact o dată
Condiție n. și s.	da (grade pare)	nu există condiție simplă
Complexitate	$O(E)$	NP-completă (problema de decizie)
Algoritm eficient	Hierholzer	nu există (în general)

Exemplu 2.1. Cele patru cazuri posibile — un graf poate fi:

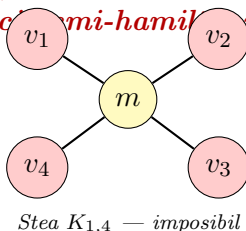
(1) **Hamiltonian**



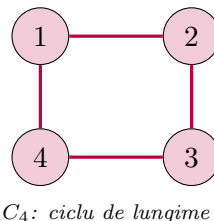
(2) **Semi-hamiltonian**



(3) **Nici hamiltonian, nici semi-hamiltonian**



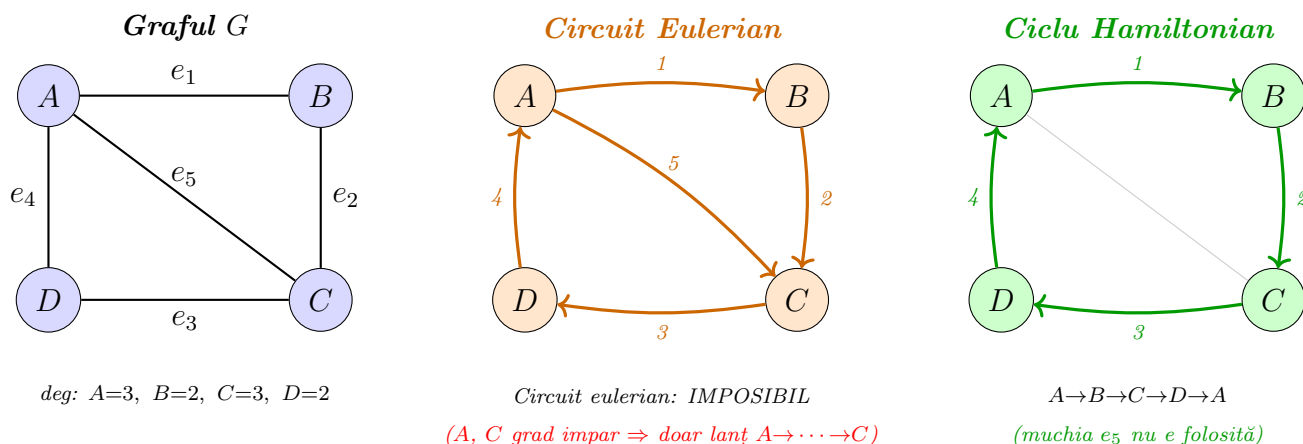
(4) **Eulerian și Hamiltonian**



- (1) Graf hamiltonian: există ciclu care trece prin a, b, c, d exact o dată.
- (2) Graf semi-hamiltonian: există lanț $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$, dar a și e nu sunt adiacente — nu se poate închide ciclul.
- (3) Graful stea $K_{1,4}$: vârful m ar trebui traversat de 4 ori — imposibil.
- (4) Ciclul C_4 este atât eulerian (toate gradele 2, pare) cât și hamiltonian.

Exemplu 2.2. Același graf — perspectivă euleriană vs. hamiltoniană

Fie graful G cu 4 vârfuri și 5 muchii:



Observăm că același graf G : nu este eulerian deoarece A și C au grad impar (există doar lanț eulerian $A \rightarrow \dots \rightarrow C$, nu circuit), dar **este hamiltonian** (ciclul $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$ folosește 4 din cele 5 muchii).

3 Condiții suficiente pentru existența ciclurilor hamiltoniene

Spre deosebire de cazul eulerian, nu există o condiție necesară și suficientă simplă. Există însă condiții **suficiente**:

Teoremă 3.1 (Ore, 1960). Fie G un graf simplu cu $n \geq 3$ vârfuri. Dacă pentru orice două vârfuri **neadiacente** $u, v \in V$:

$$\deg(u) + \deg(v) \geq n,$$

atunci G este hamiltonian.

Teoremă 3.2 (Dirac, 1952). Fie G un graf simplu cu $n \geq 3$ vârfuri. Dacă:

$$\deg(v) \geq \frac{n}{2}, \quad \forall v \in V,$$

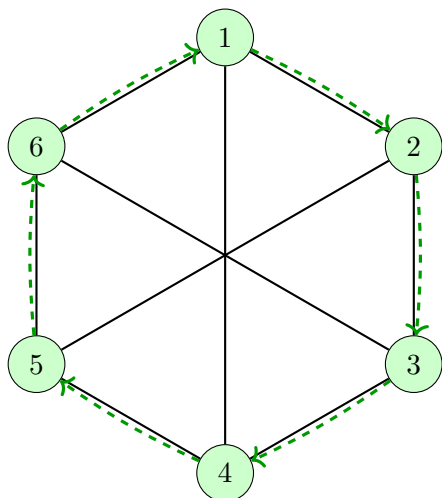
atunci G este hamiltonian.

Observație 3.1. Teorema lui Dirac este un caz particular al teoremei lui Ore. Ambele condiții sunt **suficiente, dar nu necesare**: există grafuri hamiltoniene care nu le satisfac (de exemplu C_5 , pentagoane — condiția Dirac cere $\deg \geq 2.5$, dar gradele sunt 2).

Exemplu 3.1. Ilustrarea condiției Dirac pe un graf cu $n = 6$ vârfuri.

Condiția Dirac cere $\deg(v) \geq 6/2 = 3$ pentru orice vârf.

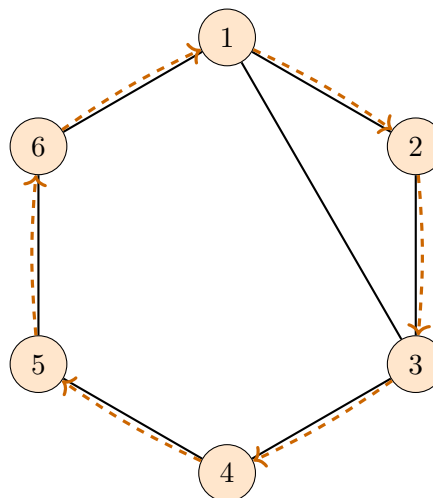
Satisface Dirac ($\deg \geq 3$)



grade: toți au grad $3 \geq 3$ ✓

Ciclu: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 1$

*NU satisface Dirac
dar e hamiltonian!*



$\deg(4) = \deg(5) = \deg(6) = 2 < 3$

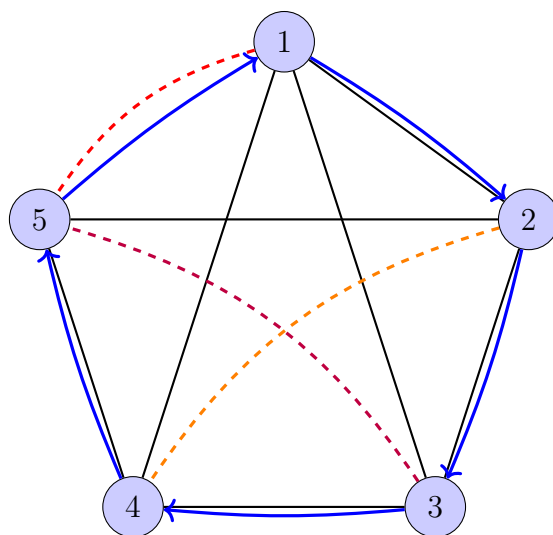
Totuși ciclu: $1 \rightarrow 2 \rightarrow \dots \rightarrow 6 \rightarrow 1$

*Concluzie: condiția lui Dirac este **suficientă** — dacă e îndeplinită, garantează existența ciclului. Dar un graf hamiltonian poate exista și fără să o îndeplinească.*

Exemplu 3.2. Ilustrarea condiției Ore — perechile neadiacente.

Fie graful cu $n = 5$ vârfuri. Condiția Ore cere că pentru orice pereche neadiacentă (u, v) : $\deg(u) + \deg(v) \geq 5$.

$$d(1) + d(5) = 3 + 2 = 5$$



$$d(2) + d(4) = 3 + 3 = 6$$

$$d(3) + d(5) = 3 + 2 = 5$$

Grade: $\deg(1) = 3$, $\deg(2) = 3$, $\deg(3) = 3$, $\deg(4) = 3$, $\deg(5) = 2$

Perechi neadiacente: $(1, 5)$, $(2, 4)$, $(3, 5)$ — toate satisfac condiția Ore

Ciclu hamiltonian: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 1$

Toate cele 3 perechi neadiacente — (1, 5), (2, 4) și (3, 5) — satisfac condiția Ore. Condiția Dirac NU e satisfăcută ($\deg(5) = 2 < 5/2 = 2.5$), dar graful este totuși hamiltonian — confirmat de condiția Ore.

```

1 def verifica_dirac(graf):
2     n = len(graf)
3     if n < 3:
4         return False, "Graful are mai puțin de 3 varfuri"
5     for v, vecini in graf.items():
6         if len(vecini) < n / 2:
7             return False, f"Varful {v} are grad {len(vecini)} < {n}/2"
8     return True, f"Condiția Dirac satisfăcută (n={n}, grad min >= {n//2})"
9
10 def verifica_ore(graf):
11     n = len(graf)
12     varfuri = list(graf.keys())
13     for i in range(n):
14         for j in range(i+1, n):
15             u, v = varfuri[i], varfuri[j]
16             if v not in graf[u]: # Neadiacente
17                 if len(graf[u]) + len(graf[v]) < n:
18                     return False, (f"Perechea ({u},{v}): "
19                                     f"{len(graf[u])}+{len(graf[v])} < {n}")
20     return True, f"Condiția Ore satisfăcută (n={n})"

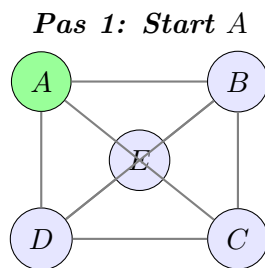
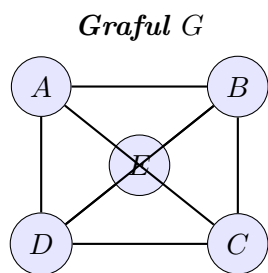
```

4 Găsirea unui ciclu hamiltonian — backtracking

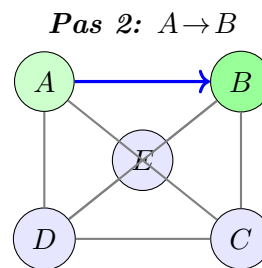
4.1 Ideea algoritmului

- (1) Pornim dintr-un vârf arbitrar (fixat pentru a evita permutările circulare).
- (2) La fiecare pas adăugăm un vârf nevizitat adiacent cu ultimul din drum.
- (3) Dacă am vizitat toate vârfurile și ultimul este adiacent cu startul $\Rightarrow$ ciclu găsit.
- (4) Altfel $\Rightarrow$ **backtrack**: eliminăm ultimul vârf și încercăm altul.

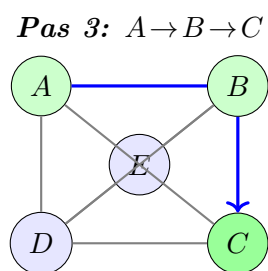
Exemplu 4.1. Evoluția backtracking-ului — urmărim fiecare pas pe un graf cu 5 vârfuri.



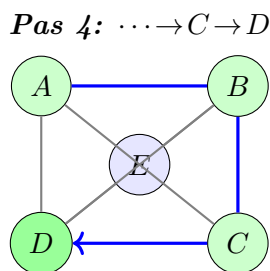
Drum: $[A]$



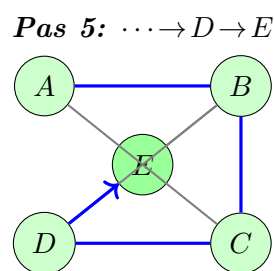
Drum: $[A, B]$



Drum: $[A, B, C]$



Drum: $[A, B, C, D]$

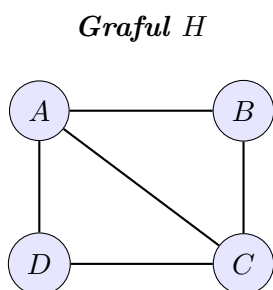


Drum: $[A, B, C, D, E]$

E adjacent cu *A*? **DA!**

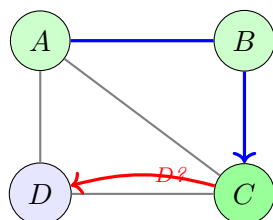
Rezultat: ciclul hamiltonian $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$ a fost găsit în 5 pași, fără nicio întoarcere (backtrack) în acest caz.

Exemplu 4.2. Backtracking cu dead end — când algoritmul trebuie să se întoarcă. Considerăm un graf unde prima alegere duce la "dead end":



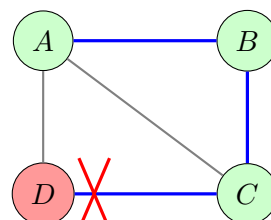
Fără muchia $B-D$

Tentativa: $A \rightarrow B \rightarrow C$



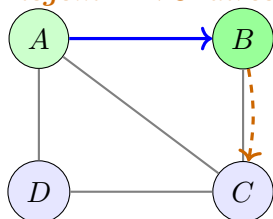
$C \rightarrow D$: OK, dar după D nu mai e vecin nevizitat al lui D !

Dead End!



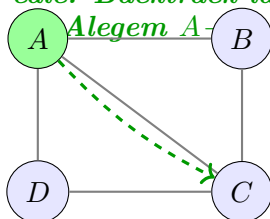
D vizitat, vecini: A, C toți deja vizitați! **Backtrack.**

Backtrack la $[A, B]$
Alegem $B \rightarrow C$ direct



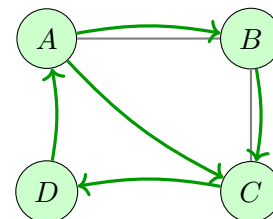
Drum curent: $[A, B]$

$A \rightarrow B \dots$ nicio cale. Backtrack la A
Alegem $A \rightarrow C$



Drum: $[A] \rightarrow$ aleasa C

CICLU GĂSIT!



$A \rightarrow C \rightarrow D \rightarrow A$?
 $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

4.2 Implementare Python

```
1 def ciclu_hamiltonian(graf):
2     """
3     Gaseste un ciclu hamiltonian prin backtracking.
4     Returneaza ciclul (lista de varfuri) sau None daca nu exista.
5     """
6     n = len(graf)
7     varfuri = list(graf.keys())
8     start = varfuri[0]
9     drum = [start]
10    vizitat = {start}
11
12    def backtrack():
13        if len(drum) == n:
14            return start in graf[drum[-1]]
15
16        for vecin in graf[drum[-1]]:
17            if vecin not in vizitat:
18                drum.append(vecin)
19                vizitat.add(vecin)
20                if backtrack():
21                    return True
22                drum.pop()
23                vizitat.remove(vecin)
24        return False
25
26    if backtrack():
27        drum.append(start)
28        return drum
29    return None
30
31
32 def lant_hamiltonian(graf):
33     """Gaseste un lant hamiltonian incercand fiecare varf ca
34     start."""
35     n = len(graf)
36
37     def backtrack(drum, vizitat):
38         if len(drum) == n:
39             return True
40         for vecin in graf[drum[-1]]:
41             if vecin not in vizitat:
42                 drum.append(vecin); vizitat.add(vecin)
43                 if backtrack(drum, vizitat):
44                     return True
45                 drum.pop(); vizitat.remove(vecin)
46         return False
47
48     for start in graf:
49         drum = [start]; vizitat = {start}
```



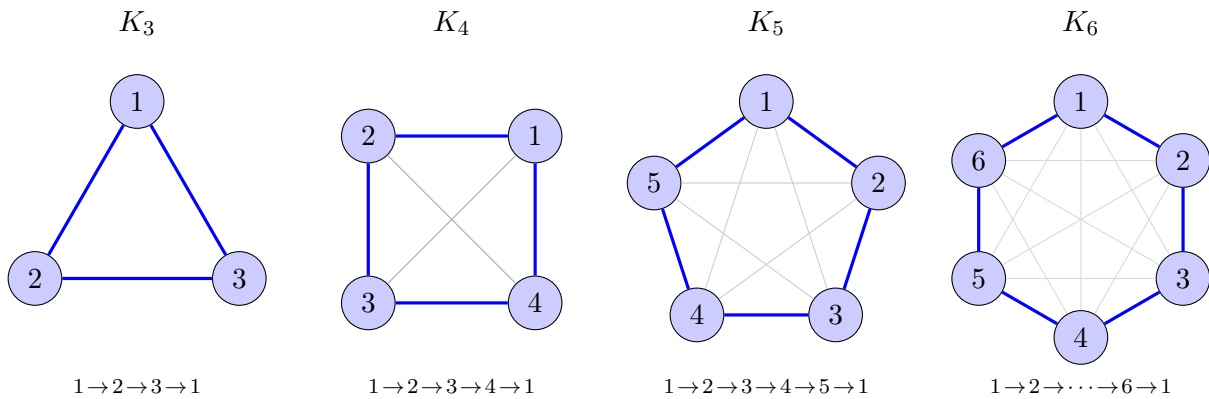
```

49         if backtrack(drum, vizitat):
50             return drum
51     return None

```

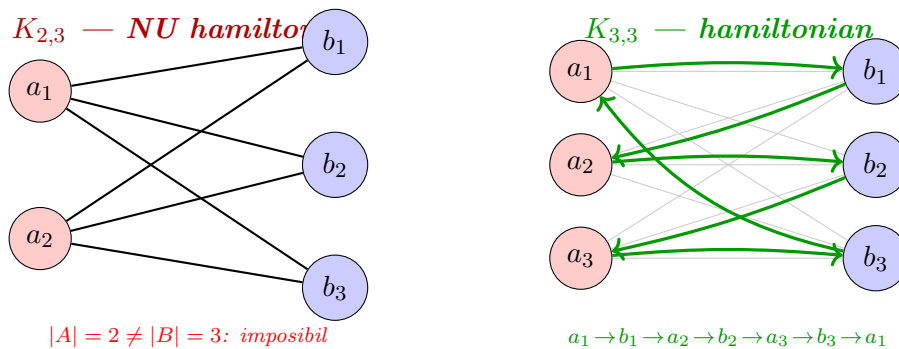
5 Grafuri celebre și proprietățile lor hamiltoniene

Exemplu 5.1. *Grafuri complete K_n — întotdeauna hamiltoniene.*



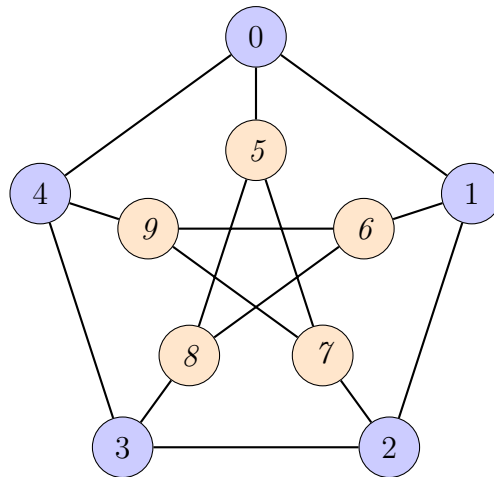
Graful complet K_n ($n \geq 3$) este întotdeauna hamiltonian, deoarece orice permutare a vârfurilor formează un ciclu hamiltonian. Numărul de cicluri hamiltoniene distincte în K_n este $\frac{(n-1)!}{2}$.

Exemplu 5.2. *Grafuri bipartite complete $K_{m,n}$ — sunt hamiltoniene dacă și numai dacă $m = n$.*



Într-un graf bipartit $K_{m,n}$, orice ciclu trebuie să alterneze între cele două mulțimi. Un ciclu hamiltonian care vizitează toți cei $m + n$ vârfuri este posibil doar dacă $m = n$ (altfel rămân vârfuri nevizitate).

Exemplu 5.3. *Graful lui Petersen — un exemplu clasic de graf 3-regulat care NU este hamiltonian.*



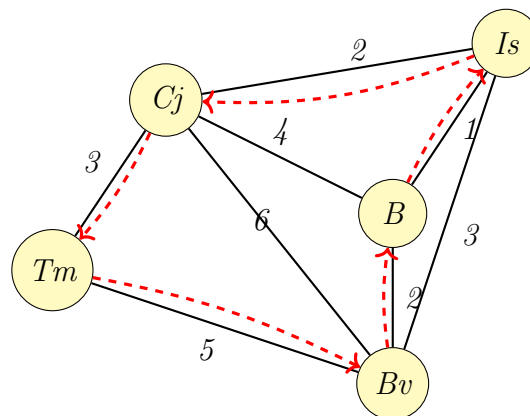
Graful lui Petersen: 3-regulat, nu are ciclu hamiltonian
(are lanț hamiltonian: $0 \rightarrow 1 \rightarrow 2 \rightarrow 7 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow 4 \rightarrow 9 \rightarrow 6$)

Graful lui Petersen este un contraexemplu celebru: este 3-regulat (fiecare vârf are grad 3), conex, este semi-hamiltonian (are lanț hamiltonian), dar **nu are ciclu hamiltonian**.

6 Aplicații practice

Exemplu 6.1. Problema comis-voiajorului (TSP) — varianta de decizie

Un agent trebuie să viziteze 5 orașe și să se întoarcă la punctul de plecare, trecând prin fiecare exact o dată.



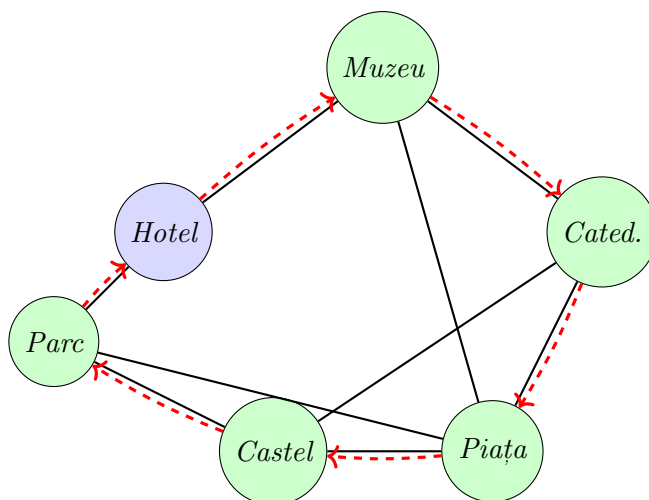
```

1 orase = {
2     'Cluj':    ['Timis', 'Buc', 'Iasi', 'Brasov'],
3     'Buc':     ['Cluj', 'Iasi', 'Brasov'],
4     'Timis':   ['Cluj', 'Brasov'],
5     'Iasi':    ['Cluj', 'Buc', 'Brasov'],
6     'Brasov': ['Timis', 'Buc', 'Iasi', 'Cluj']
7 }
8
9 tur = ciclu_hamiltonian(orase)
10 if tur:
11     print(f"Tur posibil: {' -> '.join(tur)}")
12 # Output: Cluj -> Timis -> Brasov -> Buc -> Iasi -> Cluj

```

Exemplu 6.2. Planificarea unui traseu turistic

Un turist vrea să viziteze toate atracțiile unui oraș, trecând prin fiecare exact o dată.



```

1  atractii = {
2      'Hotel':      ['Muzeu', 'Parc'],
3      'Muzeu':      ['Hotel', 'Catedrala', 'Piata'],
4      'Catedrala': ['Muzeu', 'Piata', 'Castel'],
5      'Piata':      ['Muzeu', 'Catedrala', 'Castel', 'Parc'],
6      'Castel':     ['Catedrala', 'Piata', 'Parc'],
7      'Parc':       ['Hotel', 'Piata', 'Castel']
8  }
9
10 traseu = ciclu_hamiltonian(atractii)
11 if traseu:
12     print(' -> '.join(traseu))
13 # Hotel -> Muzeu -> Catedrala -> Piata -> Castel -> Parc -> Hotel

```

Exemplu 6.3. Analiza completă a unui graf

```

1  def analiza_completa(graf, nume="Graf"):
2      n = len(graf)
3      grade = {v: len(vecini) for v, vecini in graf.items()}
4      print(f"\n{'='*45}")
5      print(f"    {nume}    (n={n} varfuri)")
6      print(f"{'='*45}")
7      print(f"Grade: {grade}")
8      ok, msg = verifica_dirac(graf)
9      print(f"Dirac: {msg}")
10     ok, msg = verifica_ore(graf)
11     print(f"Ore: {msg}")
12     ciclu = ciclu_hamiltonian(graf)
13     if ciclu:
14         print(f"Ciclu: {' -> '.join(map(str, ciclu))}")
15     else:
16         lant = lant_hamiltonian(graf)
17         if lant:

```

```
18         print(f"Lant:  {' -> '.join(map(str, lant))} (semi-  
19             hamiltonian)")  
20     else:  
21         print("Nu exista lant sau ciclu hamiltonian!")  
22  
23     # Testam pe mai multe grafuri  
24     graf_A = {1:[2,3,4,5], 2:[1,3,5], 3:[1,2,4],  
25             4:[1,3,5], 5:[1,2,4]}  
26     analiza_completa(graf_A, "Graf A")  
27  
28     graf_B = {1:[2,4], 2:[1,3], 3:[2,4], 4:[1,3]}  
29     analiza_completa(graf_B, "Graf B (ciclu C4)")  
30  
31     graf_C = {'a':['b'], 'b':['a','c'], 'c':['b','d'], 'd':['c']}  
32     analiza_completa(graf_C, "Graf C (lant P4)")
```